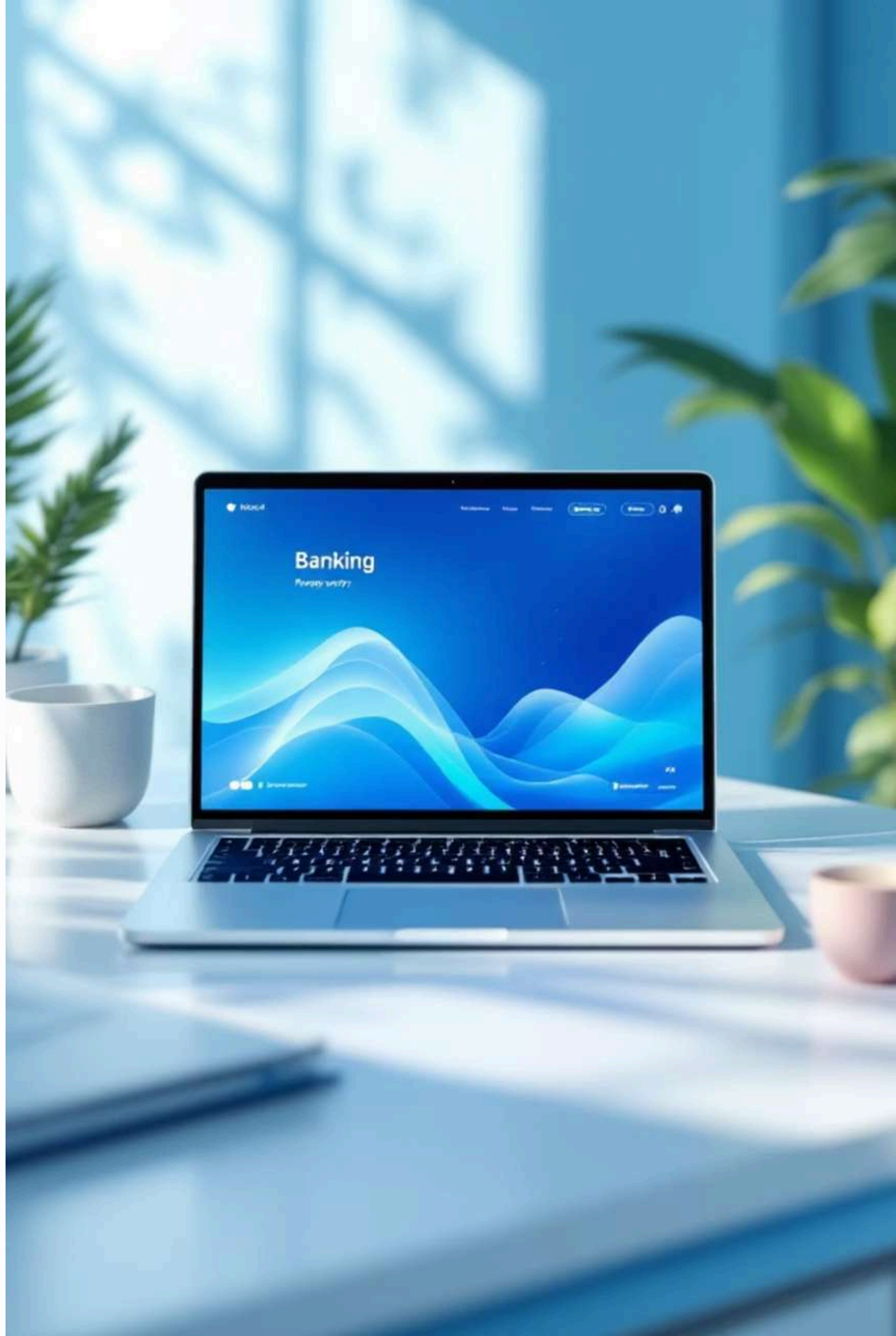
Secure Future Bank — Console Banking System (C)

A concise technical walkthrough of a Windows console-based banking application implemented in C. Focus: architecture, core modules, security (simple XOR encryption), transaction flows, mutual fund probability model, and UI design decisions for clarity and usability.



Our Team Members:

Jaideep Kalekudhiti (1262253233)

Shubham Kumar Himanshu (1262253979)

Arun Mishra (126221280)

Shivam Ramesh Lamkhade (1262252900)

Sujal Shriramjwar (1262253844)

Introduction

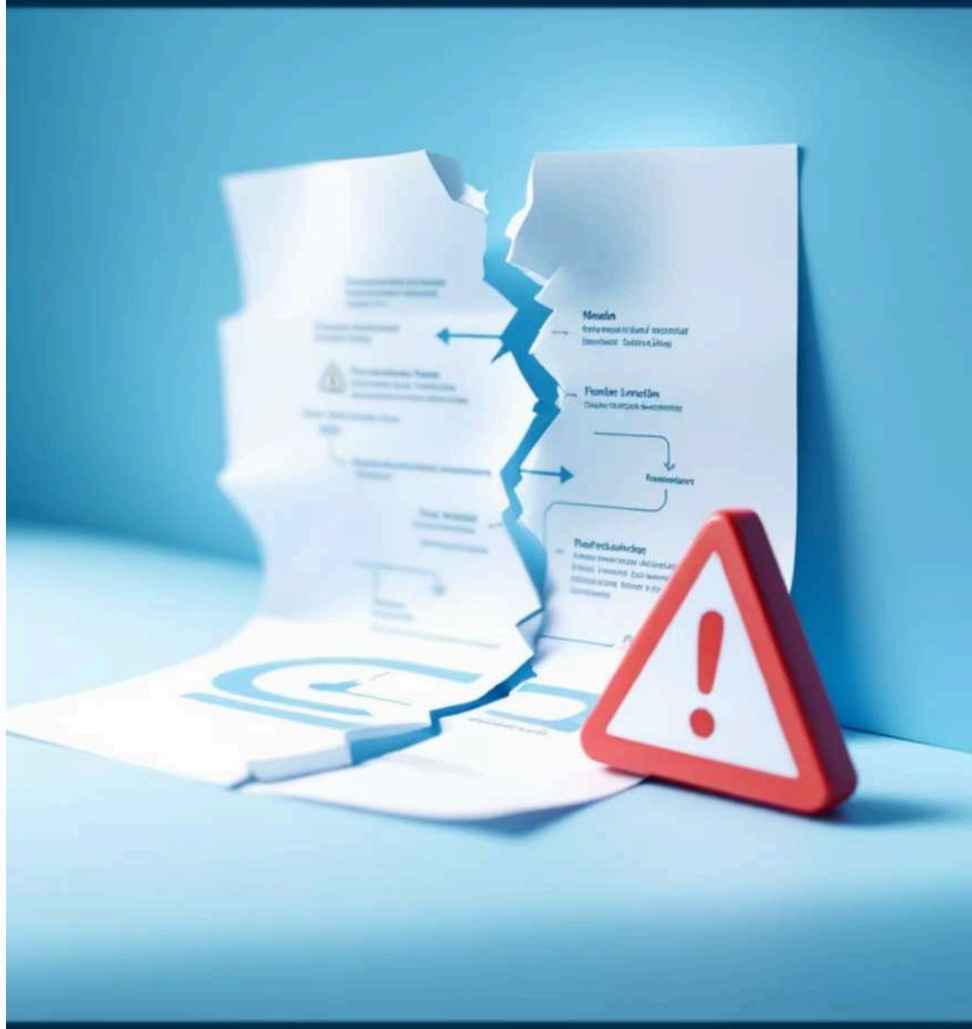
Console application written in C for Windows (uses windows.h). Supports account creation, authentication, deposits/withdrawals, transfers, loans, mutual-fund-like investments with probabilistic outcomes, and an admin overview. Data stored in-memory via struct Account; designed for learning and prototyping.

Problem Statement

Manual banking systems are time-consuming and prone to errors. There is a need for a digital system that can:

- Manage multiple accounts
- Secure user credentials
- Track financial transactions
- Provide loan and investment services
- Allow administrative monitoring

This project aims to simulate such a banking system using C programming.

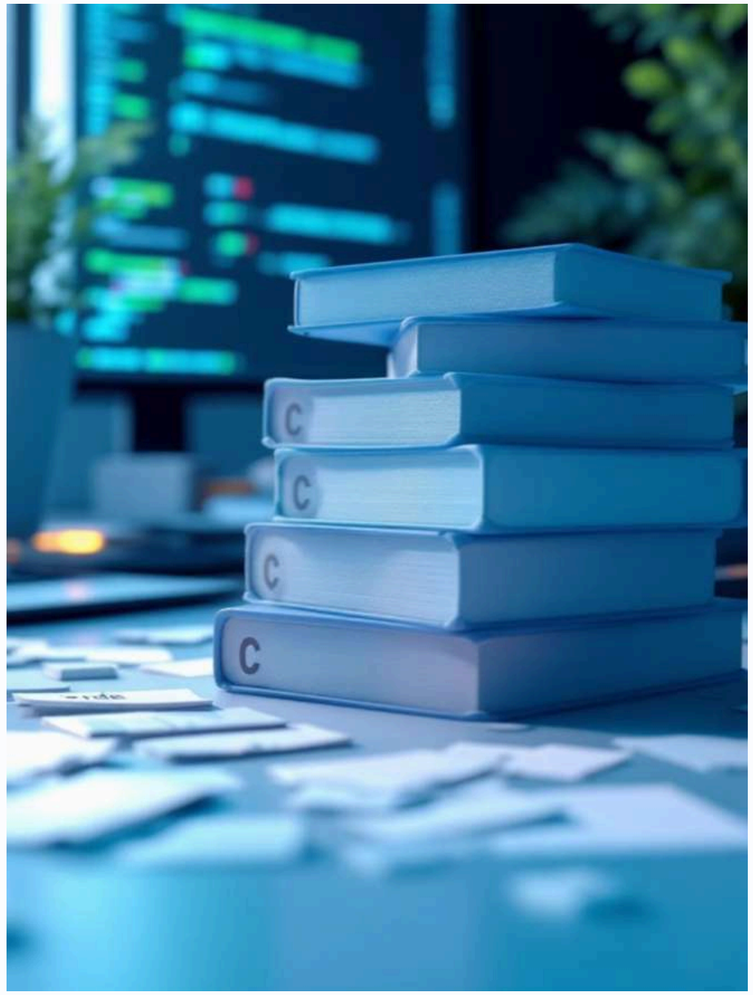


Objectives

- To simulate real-world banking operations
- To implement secure PIN authentication
- To manage deposits and withdrawals
- To implement loan calculation and repayment
- To provide mutual fund investment options
- To maintain transaction history

Architecture

This single-process, layered design separates: (1) UI (menus), (2) Services (deposit/withdraw/transfer/loan/invest), (3) Data model (struct Account array), and (4) Utilities (encryption, console helpers). No persistence layer — suitable for teaching and prototyping.



Modules

- Create Account — captures accountNo, name, sets encryptedPin, initializes fields.
- Authentication — userLogin and adminLogin (admin uses hardcoded password 1234).
- Transactions — deposit, withdraw, addTransaction(), showTransactions().
- Transfer — findAccount(), balance updates, reciprocal transaction logs.
- Loans & Mutual Funds — applyLoan(), repayLoan(), mutualFundMenu() with probabilistic returns.
- Utilities — setColor(), loadingAnimation(), header(), pauseScreen(), encryptDecrypt().

Encryption

Function: int encryptDecrypt(int pin) { int key = 5678; return pin ^ key; }

Design Notes

XOR-based obfuscation for PIN storage — reversible and extremely fast.

Security Limitations

Not cryptographically secure. Predictable key and no salting or hashing. Unsuitable for production; acceptable only for demonstrations.

Improvement Suggestions

Use salted PBKDF2/Argon2 or platform crypto APIs, secure input handling (no echo), and persist encrypted data to secure storage.

Transfer Flow

1. User enters receiver account number; application calls findAccount().
2. If found and sufficient balance, source balance decreases and destination increases.
3. Both accounts receive transaction entries via addTransaction().
4. Edge cases handled: receiver not found, insufficient funds (user feedback + pauseScreen).

Important: Currently single-threaded with immediate balance updates. For real systems, add atomic transactions, input validation, and persistence with rollback support.

Mutual Fund Probability Model

Low Risk

Rate 6% — success probability 8/10. On success: deposit + profit; on failure: retain 70% of principal.

Medium Risk

Rate 10% — success probability 7/10. Same success/failure handling as low risk but higher rate and lower success chance.

High Risk

Rate 15% — success probability 6/10. On failure retains 70% of amount; investments tracked in mutualFundInvestment field.

Notes: Randomness uses rand()%10 — uniform discrete distribution. For reproducible tests, seed control and stronger RNG recommended. Consider modeling returns with distributions (normal/log-normal) for realism.

UI Design & Usability

01

Clear Header

header() prints a consistent banner and switches console color for emphasis (SetConsoleTextAttribute).

02

Simple Menus

Text-based menus with numeric choices; immediate feedback via loadingAnimation() and pauses for readability.

03

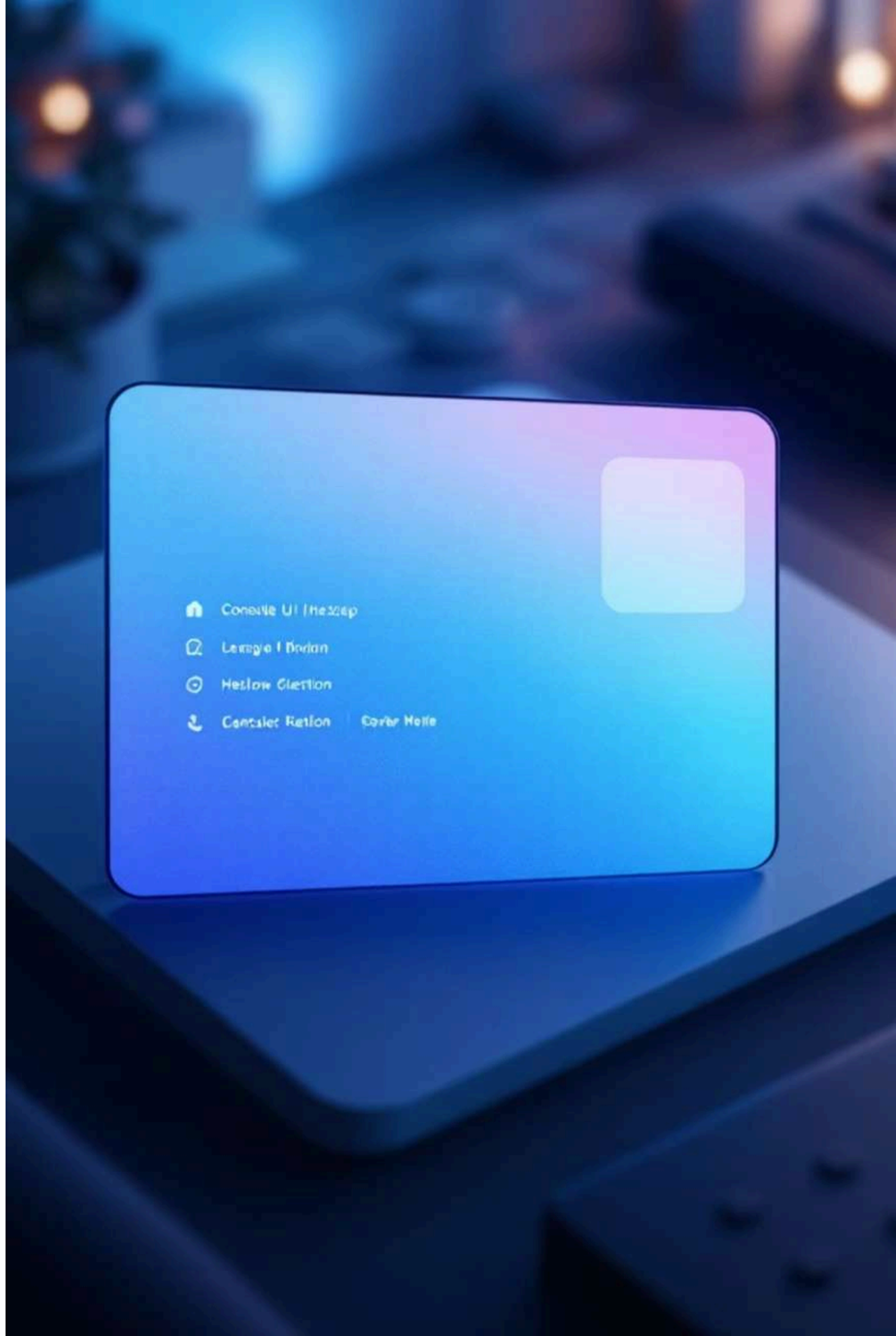
Color & Feedback

Color codes used for success (green), errors (red) and neutral info. Use consistent messages and transaction logs for traceability.

04

Improvements

Validate input robustly, avoid scanf for strings (use fgets), hide PIN input, add persistence, and modularize for testability.

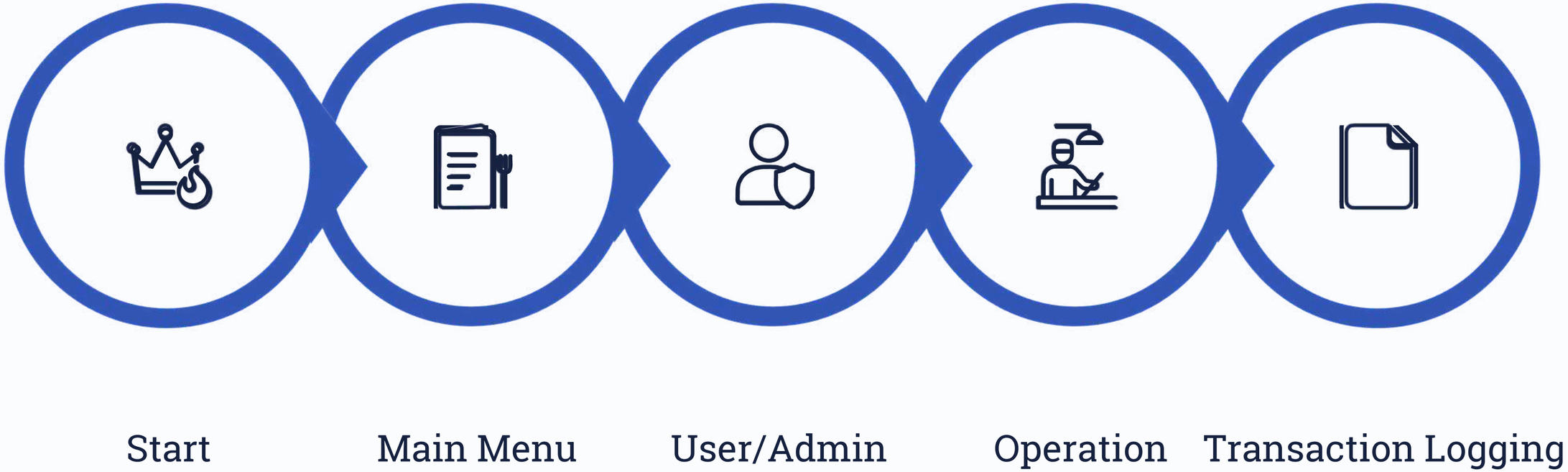


Core Banking Operations

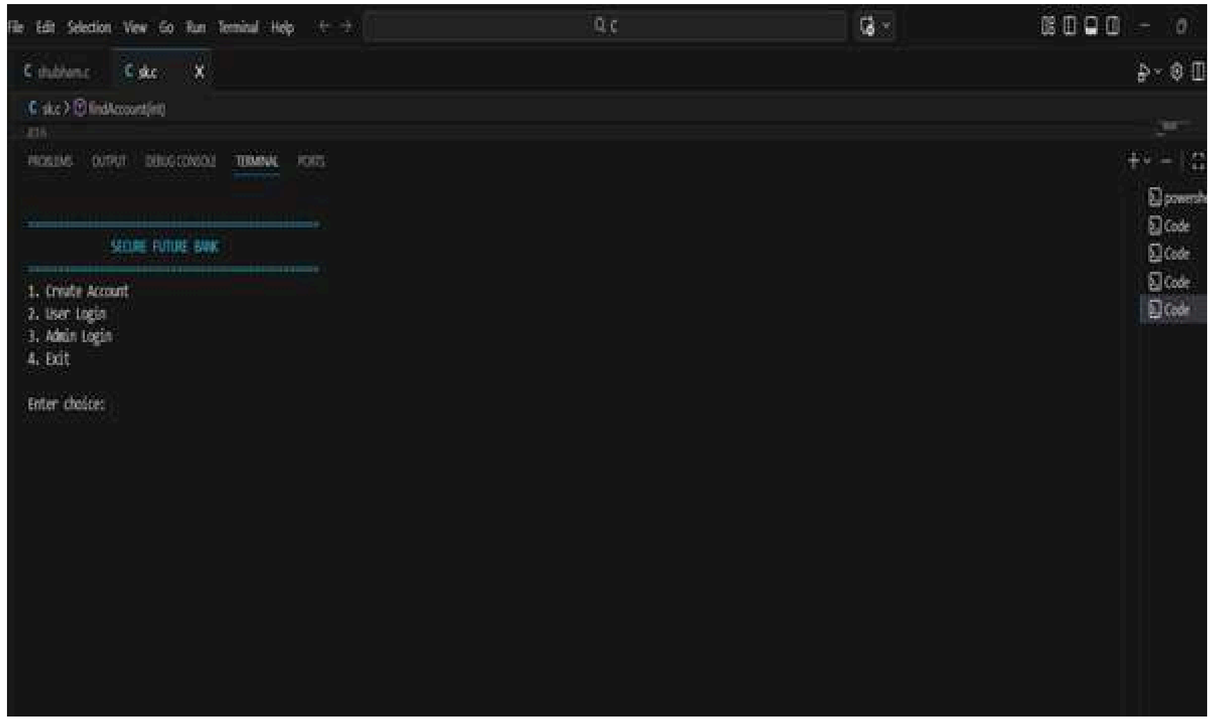
- Deposit / Withdraw — immediate balance updates and transaction logging.
- Transfer — findAccount(toAccNo), balance checks, transaction messages for both accounts.
- Loan Service — simple compound calculation by yearly loop storing loanAmount and loanTotal.
- Mutual Funds — probabilistic returns (simulated with rand()), three risk tiers and partial loss handling.

Transaction Logging & History

Each account tracks up to MAX_TRANSACTIONS messages in a fixed-size 2D char array. addTransaction appends messages; showTransactions prints numbered history. No timestamping, limited message length, no persistence across runs.



The diagram shows the program lifecycle: initialization (random seed), main menu loop, authenticated user/admin paths, action execution, and transaction recording. Use this to reason about test cases and failure points.

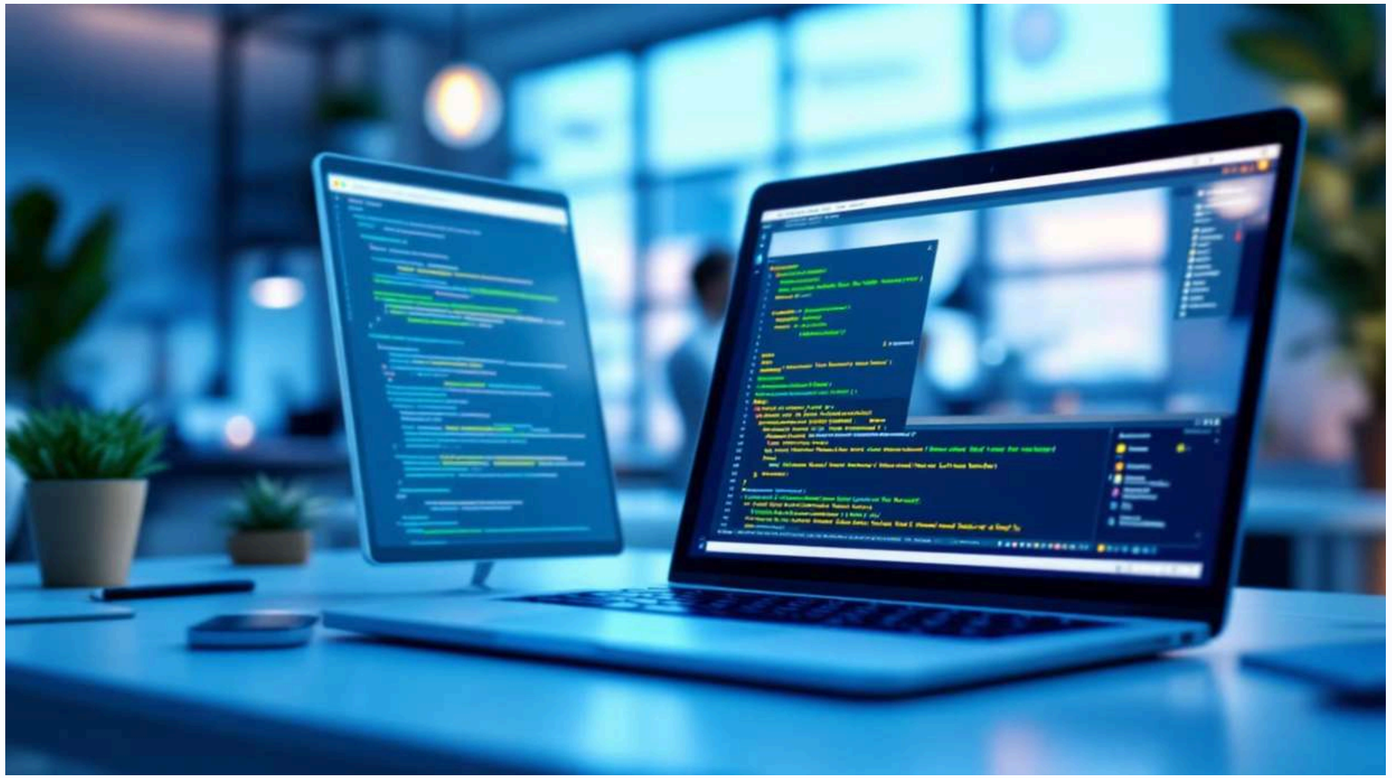


Sample Output (Representative)

Examples produced during interactive use:

- Loading... Account Created Successfully!
- Welcome, XYZ — Balance: 1500.00
- Transferred: 200.00 to AccNo 102 — Transfer Successful!
- Mutual Fund Invested: 500.00 — Investment Successful!

  You earned profit.



Testing Strategy

01

Unit Tests

Validate findAccount, encryptDecrypt (round-trip), addTransaction bounds, loan calculation accuracy.

02

Integration Tests

Simulate full user flows: create → deposit → transfer → withdraw → transaction history verification.

03

Edge Cases

Max accounts reached, transaction buffer full, insufficient balance, invalid account numbers, concurrent runs (not supported).

Thank You !!